

Sprint 7 · ML Supervisado

Regresión Lineal, Polinómica y Regularización

En este sprint aprenderás a construir modelos de regresión desde los fundamentos matemáticos hasta la implementación práctica en Python con scikit-learn. Exploraremos cómo predecir variables continuas, cómo evitar el sobreajuste y cómo aplicar técnicas de regularización para obtener modelos robustos y generalizables.

SPRINT 7

ML SUPERVISADO

SCIKIT-LEARN

Contenidos del Sprint

¿Qué aprenderemos hoy?

01

Regresión lineal simple

La idea intuitiva, la ecuación del modelo y cómo interpretamos sus coeficientes.

02

Ecuación normal vs. Gradiente

Dos formas de encontrar los parámetros óptimos del modelo.

03

Regresión polinómica

Ampliar la capacidad del modelo para capturar relaciones no lineales.

04

Sobreajuste y Regularización

Ridge y Lasso: cómo controlar la complejidad del modelo.

05

Caso práctico

Predicción del precio por noche en una plataforma de alojamiento.

La Idea de la Regresión Lineal

La regresión lineal es el modelo predictivo más fundamental del aprendizaje automático supervisado. Su objetivo es encontrar la **línea recta** (o hiperplano, en múltiples dimensiones) que mejor se ajusta a los datos, de modo que podamos predecir una variable continua **y** a partir de una o más variables de entrada **X**.

Intuición clave

Dado un conjunto de puntos en un plano, queremos trazar la línea que minimiza la distancia (error) entre las predicciones y los valores reales.

La ecuación fundamental

El modelo toma la forma:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \varepsilon$$

Donde β_0 es el intercepto, β_i son los coeficientes y ε es el error.

¿Qué minimizamos? La función de coste

Para ajustar el modelo, minimizamos la **Suma de los Errores Cuadráticos (SSE)**, también llamada función de coste o pérdida. Cuanto menor sea este valor, mejor se ajusta el modelo a los datos de entrenamiento.

$$J(\beta) = \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right)^2 = \|X\beta - y\|^2$$

Error real

La diferencia entre el valor observado y_i y el predicho $\hat{y}_i$ se llama **residuo**.

¿Por qué cuadrado?

Elevar al cuadrado penaliza errores grandes más que errores pequeños, y hace la función diferenciable en todo punto.

Métrica de evaluación

El **MSE** (Error Cuadrático Medio) y el **R²** son las métricas más usadas para evaluar la calidad del ajuste.

Ecuación Normal vs. Descenso del Gradiente

Existen dos estrategias principales para encontrar los coeficientes β que minimizan la función de coste. Cada una tiene sus ventajas según el tamaño y la dimensionalidad de los datos.



Ecuación Normal

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

Solución directa y exacta. No requiere iteraciones ni ajustar tasa de aprendizaje. Sin embargo, invertir $X^T X$ tiene un coste de $O(n^3)$, lo que la hace impráctica cuando hay muchas columnas (características).

-  Solución exacta en un solo paso
-  Lento con muchas variables ($n > 10.000$)



Descenso del Gradiente

$$\beta := \beta - \alpha \nabla_{\beta} J(\beta)$$

Método iterativo que actualiza los parámetros paso a paso en la dirección de mayor descenso de la función de coste. Escalable a grandes conjuntos de datos.

-  Funciona con millones de muestras
-  Requiere ajustar el learning rate α

Supuestos de la Regresión Lineal

Para que los coeficientes estimados sean **insesgados y eficientes** (teorema de Gauss-Markov), el modelo debe cumplir una serie de condiciones sobre los datos y los errores. Violar estos supuestos no siempre invalida el modelo, pero sí afecta a la interpretación y fiabilidad de las predicciones.



Linealidad

La relación entre X e y debe ser lineal en los parámetros. Verificar con gráficos de residuos.



Normalidad de errores

Los residuos deben seguir una distribución normal. Se comprueba con Q-Q plots o la prueba Shapiro-Wilk.



Independencia

Los errores deben ser independientes entre sí. Especialmente importante en series temporales.



Homocedasticidad

La varianza de los errores debe ser constante a lo largo de todos los valores de X (sin patrones de embudo).

Interpretación de los Coeficientes

Los coeficientes β son el corazón del modelo: nos dicen cuánto cambia la variable objetivo por cada unidad de cambio en una variable predictora, *manteniendo el resto constantes*. Esta interpretación es lo que hace la regresión lineal tan valiosa más allá de la predicción.

Coeficiente $\beta_1 = 45$

Si el modelo predice el precio de un alojamiento y $\beta_1 = 45$ para la variable "número de habitaciones", esto significa: *cada habitación adicional aumenta el precio en 45 € de media*, con el resto de variables fijas.

Signos e importancia

- **β positivo:** la variable predictora tiene relación directa con y .
- **β negativo:** relación inversa. Más X implica menos y .
- **$\beta \approx 0$:** esa variable apenas contribuye al modelo.
- **p-valor:** indica si el coeficiente es estadísticamente significativo.

 Con variables en distintas escalas, escala los datos antes de comparar la magnitud de los coeficientes.



Regresión Polinómica

Cuando la relación entre variables no es lineal, podemos **eleva**r las variables de **entrada a potencias superiores** para capturar curvas y patrones más complejos. Esto se llama regresión polinómica, y técnicamente sigue siendo un modelo lineal en sus parámetros.

De la Recta a la Curva

La regresión polinómica añade términos de mayor orden como $\mathbf{x^2}$, $\mathbf{x^3}$, ... al modelo, transformando el espacio de características sin cambiar el algoritmo de regresión subyacente. El grado del polinomio es un **hiperparámetro clave** que controla la flexibilidad del modelo.

Modelo lineal (grado 1)

$$y = \beta_0 + \beta_1 x$$

Modelo cuadrático (grado 2)

$$y = \beta_0 + \beta_1 x + \beta_2 x^2$$

Grado n

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_n x^n$$

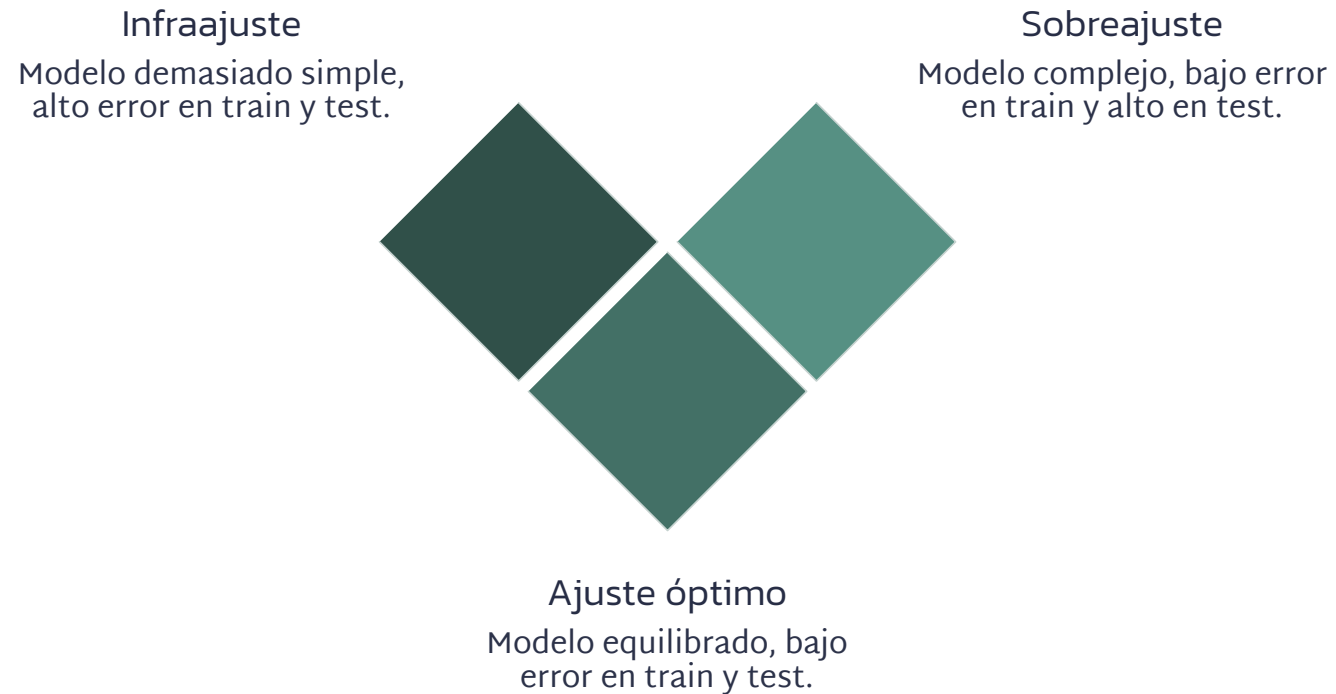
En scikit-learn

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ('poly', PolynomialFeatures(degree=2)),
    ('lr', LinearRegression())
])
pipe.fit(X_train, y_train)
```

Sobreajuste: El Gran Enemigo del ML

Un modelo que aprende **demasiado bien** los datos de entrenamiento —incluyendo el ruido— falla a la hora de generalizar a datos nuevos. Este fenómeno se llama **sobreajuste (overfitting)** y es uno de los problemas más frecuentes en aprendizaje automático.



La clave está en el **equilibrio sesgo-varianza**: modelos simples tienen alto sesgo (underfitting), modelos complejos tienen alta varianza (overfitting).

Diagnóstico visual

Curvas de Aprendizaje

Las **curvas de aprendizaje** son una herramienta esencial para diagnosticar si un modelo sufre sobreajuste o infraajuste. Muestran cómo evoluciona el error en entrenamiento y validación a medida que aumentamos el tamaño del conjunto de entrenamiento o la complejidad del modelo.

Señales de sobreajuste

- Error muy bajo en entrenamiento
- Error muy alto en validación
- Gran brecha entre ambas curvas
- El modelo memoriza el ruido de los datos

Señales de infraajuste

- Error alto tanto en train como en val
- Las curvas convergen pronto y a un valor alto
- El modelo es demasiado simple para los datos

Modelo bien calibrado

- Error de validación bajo y estable
- Pequeña diferencia entre train y val
- Las curvas convergen a un valor razonable



Regularización: Controlando la Complejidad

La regularización es una técnica que **añade una penalización a la función de coste** proporcional a la magnitud de los coeficientes. De este modo, el modelo se ve "forzado" a ser más sencillo, reduciendo el sobreajuste sin necesidad de reducir el número de variables manualmente.

$$J_{reg}(\beta) = \underbrace{\|X\beta - y\|^2}_{\text{Error de ajuste}} + \underbrace{\lambda \cdot \Omega(\beta)}_{\text{Penalización}}$$

El parámetro **λ (lambda)** controla el equilibrio: $\lambda=0$ equivale a regresión sin regularización; λ muy grande produce coeficientes cercanos a cero.

Ridge vs. Lasso: Dos Enfoques

Las dos técnicas de regularización más usadas difieren en el tipo de penalización que aplican sobre los coeficientes, lo que produce comportamientos distintos y hace que cada una sea más adecuada según el contexto.

Ridge (L2)

$$\lambda \sum_j \beta_j^2$$

Penaliza el **cuadrado** de los coeficientes. Los reduce hacia cero pero **nunca los elimina del todo**. Ideal cuando todas las variables son potencialmente relevantes. Especialmente útil ante multicolinealidad.

Lasso (L1)

$$\lambda \sum_j |\beta_j|$$

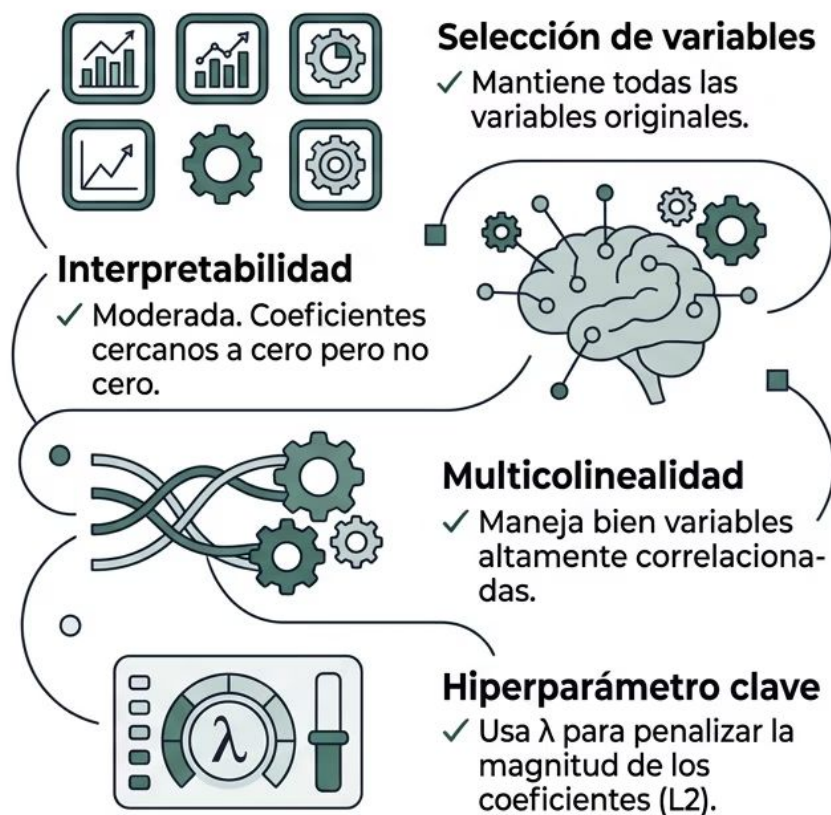
Penaliza el **valor absoluto** de los coeficientes. Puede llevarlos exactamente a cero, realizando una **selección automática de variables**. Produce modelos más interpretables y dispersos (sparse).



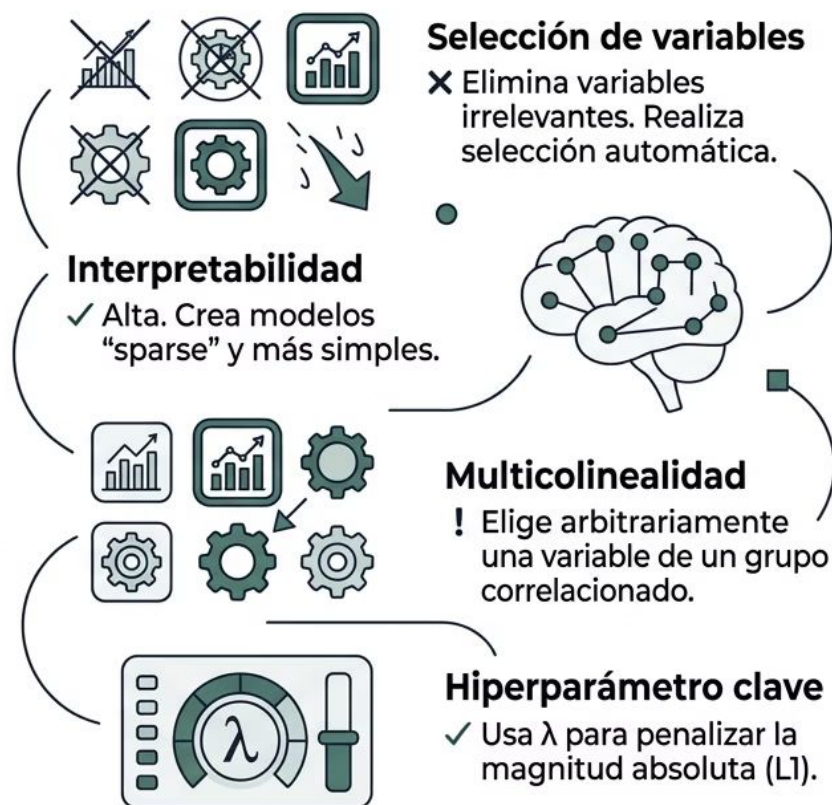
❏ Elastic Net combina ambas penalizaciones (L1 + L2) para obtener lo mejor de cada enfoque.

¿Cuándo usar Ridge y cuándo Lasso?

RIDGE REGRESSION



LASSO REGRESSION



En la práctica, se recomienda probar ambos con **validación cruzada** para seleccionar λ óptimo usando `RidgeCV` y `LassoCV` de `scikit-learn`.

Implementación en scikit-learn

scikit-learn ofrece una API uniforme y consistente para todos los modelos de regresión. A continuación, el flujo completo desde la carga de datos hasta la evaluación del modelo.

```
from sklearn.linear_model import Ridge, Lasso, LinearRegression
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import mean_squared_error, r2_score
```

```
# 1. Dividir datos
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# 2. Pipeline con escalado + Ridge
```

```
pipe_ridge = Pipeline([
    ('scaler', StandardScaler()),
    ('model', Ridge(alpha=1.0)) # alpha = lambda
])
```

```
# 3. Entrenar y evaluar
```

```
pipe_ridge.fit(X_train, y_train)
y_pred = pipe_ridge.predict(X_test)
```

```
print(f"MSE : {mean_squared_error(y_test, y_pred):.2f}")
```

```
print(f"R2 : {r2_score(y_test, y_pred):.3f}")
```

 Recuerda: en scikit-learn el parámetro de regularización se llama `alpha`, no `lambda`.

Selección de λ con Validación Cruzada

Elegir el valor correcto de λ es fundamental. Demasiado pequeño y el modelo sobreajusta; demasiado grande y el modelo infraajusta. La validación cruzada nos permite encontrar el valor óptimo de forma automática y robusta.

RidgeCV y LassoCV

```
from sklearn.linear_model import RidgeCV, LassoCV
import numpy as np

alphas = np.logspace(-3, 3, 100)

ridge_cv = RidgeCV(alphas=alphas, cv=5)
ridge_cv.fit(X_train_scaled, y_train)

print(f"Mejor alpha: {ridge_cv.alpha_:.4f}")

lasso_cv = LassoCV(alphas=alphas, cv=5, max_iter=10000)
lasso_cv.fit(X_train_scaled, y_train)
print(f"Mejor alpha: {lasso_cv.alpha_:.4f}")
print(f"Coefs. no nulos: {(lasso_cv.coef_ != 0).sum()}")
```

¿Cómo funciona?

La validación cruzada con **k pliegues** (cv=5) divide el conjunto de entrenamiento en 5 partes. Para cada valor de α , entrena en 4 partes y valida en la quinta, rotando. Devuelve el α con menor error de validación promedio.

- Usa `np.logspace` para explorar órdenes de magnitud
- Siempre escala los datos antes de regularizar
- El atributo `.alpha_` devuelve el mejor valor encontrado

Caso Práctico: Predicción del Precio por Noche

Aplicamos todo lo aprendido a un problema real: **predecir el precio por noche de un alojamiento** a partir de sus características. Este tipo de modelo es el núcleo de los sistemas de fijación de precios dinámicos usados en plataformas como Airbnb.

CASO REAL

REGRESIÓN

AIRBNB-STYLE



Variables del Conjunto de Datos

El conjunto de datos incluye características del alojamiento, del anfitrión y de la ubicación. El objetivo es predecir **price** (precio por noche en euros).

Variables numéricas

- `accommodates`: número de huéspedes
- `bathrooms`: número de baños
- `bedrooms`: número de habitaciones
- `beds`: número de camas
- `review_scores_rating`: puntuación media
- `number_of_reviews`: total de reseñas
- `host_listings_count`: propiedades del anfitrión

Variables categóricas

- `room_type`: tipo de habitación (Entire, Private, Shared)
- `neighbourhood`: barrio de la ciudad
- `property_type`: tipo de propiedad

Target

- `price`: precio por noche en € (variable continua)

Pipeline Completo: del Dato al Modelo

A continuación, el flujo completo del caso práctico: limpieza, ingeniería de características, modelado con regularización y evaluación comparativa.

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LassoCV
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

df = pd.read_csv('listings.csv')
df['price'] = df['price'].str.replace('[$,]', '', regex=True).astype(float)
df = df.dropna(subset=['price', 'bedrooms', 'bathrooms'])

num_cols = ['accommodates', 'bedrooms', 'bathrooms', 'beds',
            'review_scores_rating', 'number_of_reviews']
cat_cols = ['room_type', 'neighbourhood_cleansed']

X = df[num_cols + cat_cols]
y = np.log1p(df['price']) # log-transform para estabilizar varianza

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), num_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)
])

pipe = Pipeline([
    ('prep', preprocessor),
    ('model', LassoCV(cv=5, max_iter=10000))
])

pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
print(f"R² : {r2_score(y_test, y_pred):.3f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_pred)):.3f}")
```

Cierre del Sprint 7

Resumen y Puntos Clave

Hemos recorrido el camino completo de la regresión, desde la intuición hasta la implementación. Estos son los conceptos esenciales que debes llevarte:

- 1** La regresión lineal busca la recta óptima
Minimizando el error cuadrático mediante la ecuación normal (exacta) o el descenso del gradiente (escalable).
- 2** Los supuestos importan
Linealidad, independencia, normalidad y homocedasticidad de los errores son fundamentales para una interpretación válida.
- 3** El grado del polinomio controla la flexibilidad
A mayor grado, mayor capacidad de ajustar curvas, pero también mayor riesgo de sobreajuste.
- 4** Ridge conserva; Lasso selecciona
La regularización L2 encoge los coeficientes, la L1 puede llevarlos a cero. Siempre calibra λ con validación cruzada.

📅 Próximo sprint: árboles de decisión, Random Forest y Gradient Boosting. ¡Seguimos construyendo! 🚀